Task 01 - Coding Agent Basics: Suggestion vs Verified Result

Handbook reference: Page 1 of 10 | Report date: 2026-10-13 | Folder: `task-01-coding-agent-basics/`

1. Concept from the handbook

A coding agent is a supervised engineering assistant: model + instructions + context + authorised tools in an execution loop. It is not a chatbot, not autocomplete and not a human developer. The key rule of the page: a model **proposing** a command and the application **executing** it are different things. 'I ran the test' is a verified claim only if an observable tool result exists.

2. What was built

An **evidence recorder** (`run_check`) that really executes a command and stores the command, return code, output, time and exact revision (content hash). A **claim judge** then labels every claim as VERIFIED, FAILED, BLOCKED/timeout or NOT VERIFIED (no execution evidence).

3. Behaviours demonstrated

- Claim without evidence -> 'NOT VERIFIED'
- Attempt that was blocked / timed out is NOT a completed action
- A failing run is reported as FAILED, never as success
- Evidence is tied to an exact revision (hash of the code)

4. Files in this task

`demo.py`, `evidence.py`, `sample_project/tasklist.py`, `sample_project/test_tasklist.py`, `test_evidence.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

evidence.py

```python
"""Task 01 - Suggestion vs verified result.

A model PROPOSING a command is not the same as the application EXECUTING it.
"I ran the tests" is only a verified claim if an observable tool result exists.
This module records execution evidence and checks claims against it.
"""
from __future__ import annotations

import hashlib
import subprocess
import sys
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import List, Optional

VERIFIED = "VERIFIED"
FAILED = "FAILED (attempt ran, result is a failure)"
NOT_EXECUTED = "NOT VERIFIED (no execution evidence)"
BLOCKED = "NOT VERIFIED (command blocked / timed out)"


@dataclass
class Evidence:
    """Observable result of a command that was really executed."""
    command: List[str]
    returncode: Optional[int]          # None => never finished (timeout / blocked)
    stdout: str = ""
    stderr: str = ""
    started_at: str = field(default_factory=lambda: datetime.now(timezone.utc).isoformat(timespec="seconds"))
    revision: str = ""                 # exact code revision the evidence belongs to
    note: str = ""

    @property
    def finished(self) -> bool:
        return self.returncode is not None

    @property
    def passed(self) -> bool:
        return self.returncode == 0


def revision_of(path: Path) -> str:
    """Content hash of every .py file => 'exact revision' when git is not used."""
    h = hashlib.sha256()
    for p in sorted(Path(path).rglob("*.py")):
        h.update(p.name.encode())
        h.update(p.read_bytes())
    return h.hexdigest()[:12]


def run_check(command: List[str], cwd: Path, timeout: int = 30) -> Evidence:
    """Actually execute a command and capture what happened."""
    rev = revision_of(cwd)
    try:
        cp = subprocess.run(command, cwd=cwd, capture_output=True, text=True, timeout=timeout)
        return Evidence(command, cp.returncode, cp.stdout, cp.stderr, revision=rev)
    except subprocess.TimeoutExpired:
        return Evidence(command, None, revision=rev, note=f"timeout after {timeout}s")
    except (FileNotFoundError, PermissionError) as exc:
        return Evidence(command, None, revision=rev, note=f"blocked: {exc}")


@dataclass
class Claim:
    text: str
    evidence: Optional[Evidence] = None


def judge(claim: Claim) -> str:
    """Return the honest status of a claim."""
    ev = claim.evidence
    if ev is None:
        return NOT_EXECUTED
    if not ev.finished:
        return BLOCKED
    return VERIFIED if ev.passed else FAILED


def render_report(claims: List[Claim]) -> str:
```

Main implementation: evidence.py (part 1/2)

```
    lines = []
    for i, c in enumerate(claims, 1):
        status = judge(c)
        lines.append(f"[{i}] claim : {c.text}")
        lines.append(f"    status: {status}")
        if c.evidence:
            e = c.evidence
            shown = [Path(e.command[0]).name] + e.command[1:]
            lines.append(f"    cmd   : {' '.join(shown)}")
            lines.append(f"    rc    : {e.returncode}   revision: {e.revision}   {e.note}".rstrip())
            tail = (e.stderr or e.stdout).strip().splitlines()[-1:] or [""]
            lines.append(f"    output: {tail[0]}")
        lines.append("")
    return "\n".join(lines)


PYTHON = sys.executable
```

Main implementation: evidence.py (part 2/2)

demo.py

```
"""Demo: three claims an agent might make, judged by evidence."""
from pathlib import Path

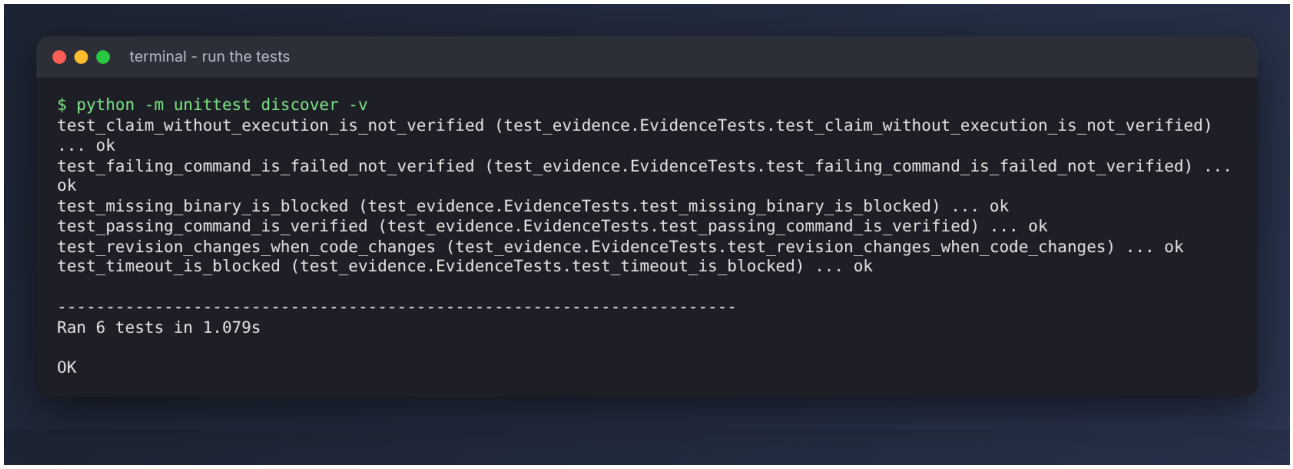
from evidence import Claim, PYTHON, render_report, run_check

project = Path(__file__).parent / "sample_project"

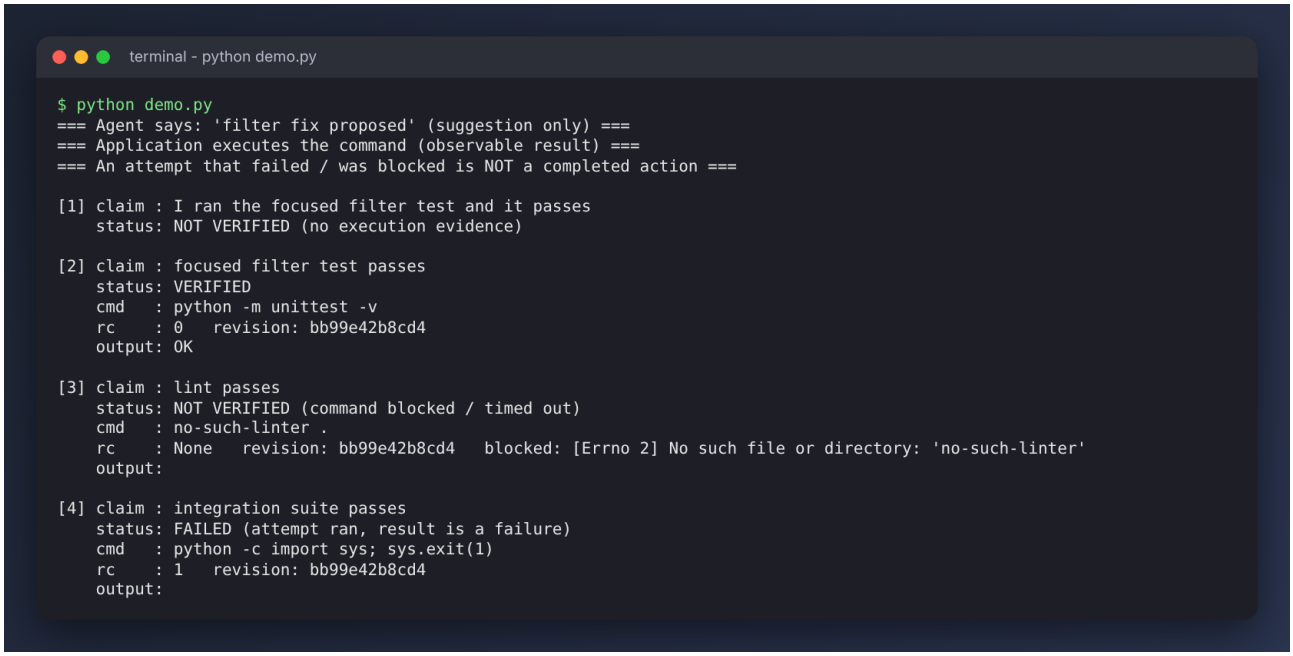
print("=== Agent says: 'filter fix proposed' (suggestion only) ===")
claims = [
    Claim("I ran the focused filter test and it passes"),            # no evidence at all
]
print("=== Application executes the command (observable result) ===")
claims.append(Claim("focused filter test passes",
                    run_check([PYTHON, "-m", "unittest", "-v"], project)))
print("=== An attempt that failed / was blocked is NOT a completed action ===")
claims.append(Claim("lint passes", run_check(["no-such-linter", "."], project)))
claims.append(Claim("integration suite passes",
                    run_check([PYTHON, "-c", "import sys; sys.exit(1)"], project)))
print()
print(render_report(claims))
```

Demo script

6. Demo and test screenshots



Test run (real output)



Demo output (python demo.py)

7. Evidence-backed report

Change	Coding Agent Basics: Suggestion vs Verified Result
Revision tested	content hash `0fc202efbe9a` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 6 tests in 1.097s
Checks not run	Real LLM calls (the demo uses fixed example claims)
Known limitations	Revision = content hash of .py files; in a git repo use the commit SHA (see Task 07).
Next required action	Human review, then CI on the pull request

8. Practical takeaway

You can delegate work to an agent, never accountability. Keep scope + evidence + human judgement together.